Verslag project

Opdracht

Als project hebben we ons gebaseerd op een masterproef '*NEURAL NETWORK BASED INDOOR POSITIONING WITH ULTRASOUND SIGNALLING, ENABLED BY AN AUTOMATED DATA COLLECTION ROBOT*'. Hierbij is het de bedoeling dat we in een ruimte een positie kunnen bepalen van een robot door middel van een akoestisch signaal door te sturen. Dit signaal zal zowel een rechtstreeks signaal naar de ontvanger zijn, alsook de weerkaatste signalen via muren, en eventueel andere objecten. Door deze signalen in te lezen zal men afhankelijk van de grootte van de signalen telkens een soort fingerprint hebben. Deze fingerprint vergelijken we met een database van alle mogelijke posities om na te gaan welke hiermee het beste overeenkomt. Hierbij zal de positie waarvan de waarden het best overeen komen de positie zijn waar onze robot zich het dichtst bij bevindt.

Een schets van de situatie zien we hieronder. We zien de ruimte met daarin verschillende posities (blauwe cirkels) waarvan we de positie en bijhorende signalen hebben opgeslagen in onze LUT. In de linkeronderhoek zit de zender die de signalen genereert. Onze robot ontvangt hiervan een direct signaal, en meerdere weerkaatste signalen. Op basis hiervan zullen we dus detecteren waar onze robot zich in de ruimte bevindt.

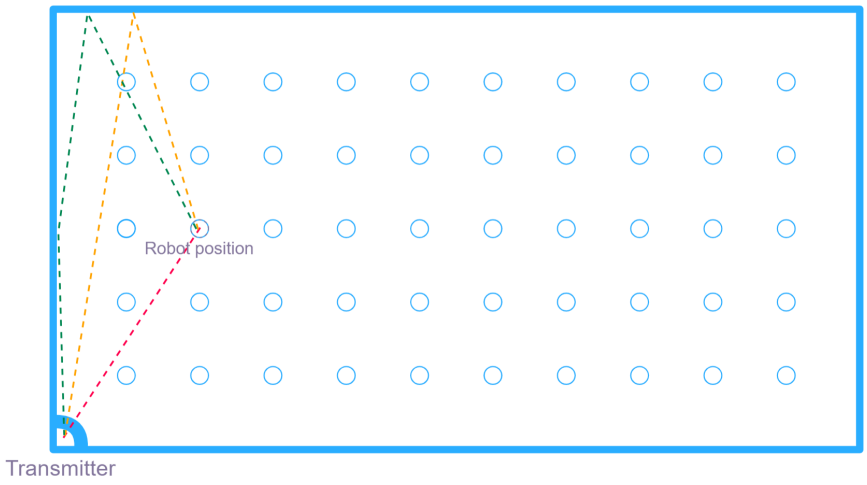
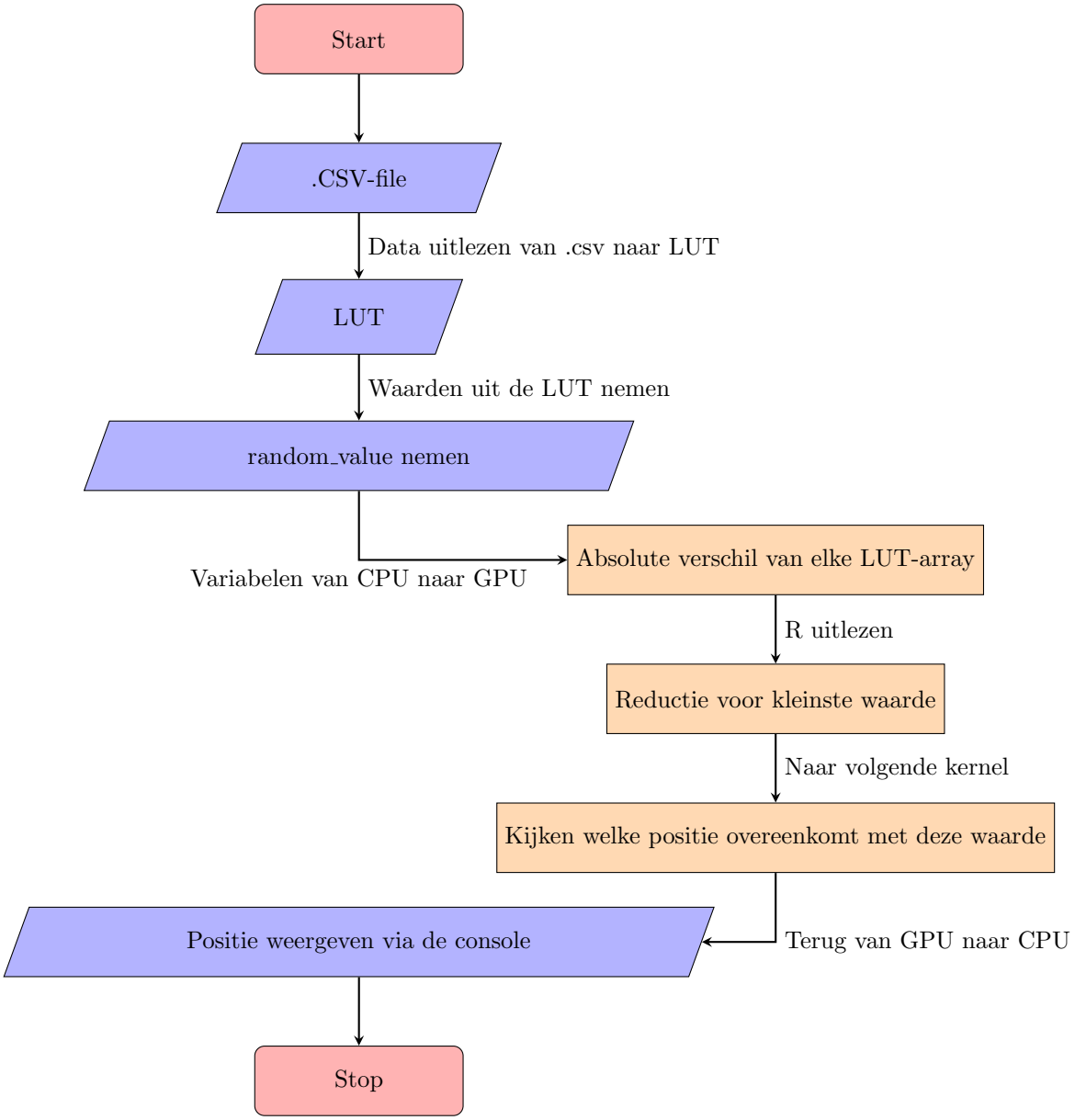


Figure 1: Situatieschets

Werkwijze

Eerst hebben we een lookuptable (verder LUT) gegenereerd van allemaal random waarden die bestaat uit verticaal de verschillende posities en horizontaal het aantal weerkaatsingen (de groottes van de weerkaatsingen waren willekeurig gekozen). Vervolgens werd hieruit één rij genomen die zal dienen als de positie. Hierna werd gekeken in de GPU of we deze random array terugvonden in de LUT, als dit zo was werd aan de output de positie meegegeven waarop de random array zich bevinden. Eens dit lukte werd gekeken of dit ook werkte voor een ander aantal weerkaatsingen (van drie naar vier). Tenslotte werd de code aangepast zodat wanneer we geen exacte overeenkomst van de willekeurige array met de LUT hebben, we de positie nemen met het kleinste verschil qua waarden. Hiervoor werd eerst een extra kernel aangemaakt waarop reduction wordt toegepast voor het kleinste verschil te weten waarbij deze waarde dan werden meegegeven aan de volgende kernel om te kijken op welke positie zich dit bevindt. Als laatste werd via een script een LUT aangemaakt als .CSV file om zo allemaal verschillende, en realistische waarden te bekomen.

Flowchart


Start
.CSV-file
Data uitlezen van .csv naar LUT
LUT
Waarden uit de LUT nemen
random_value nemen
Variabelen van CPU naar GPU
Absolute verschil van elke LUT-array
R uitlezen
Reductie voor kleinste waarde
Naar volgende kernel
Kijken welke positie overeenkomt met deze waarde
Terug van GPU naar CPU
Positie weergeven via de console
Stop

Code

De code bestaat uit verschillende delen. Als eerste het aanmaken van de LUT met alle waarden welke verkregen worden door de geïmporteerde .CSV file. Vervolgens wordt deze geconverteerd naar een 1D array om deze later makkelijker te kunnen gebruiken. Uit deze LUT wordt een willekeurige waarde uitgehaald. Deze waarde zal dienen als de zogenaamde huidige positie, wat betekent dat we deze waarde moeten terugvinden uit de LUT. Vervolgens worden alle variabelen en pointers aangemaakt om de data over te zetten van de CPU naar de GPU. In de GPU gaan we eerst de kernel 'getMinValue' gebruiken. Deze zal een bewerking uitvoeren die het absolute verschil weergeeft van de willekeurig gekozen array en elke lijn uit de LUT. Deze uitkomst plaatsen we in een nieuwe array R. Op R voeren we dan reduction uit om te bepalen wat het kleinst mogelijke verschil is tussen de random gekozen array en de input op een bepaalde positie. Deze waarde geven we dan mee aan de volgende kernel. Hier gaan we terug dezelfde bewerking uitvoeren als de vorige kernel en kijken welke array van de LUT gelijk is aan de minimale waarde die we hebben berekent uit de vorige kernel. Als deze overeenstemmen weten we op welke lijn uit de LUT de overeenkomst zit, en zal dus deze positie het dichtst bij onze huidige positie liggen. Deze lijn (of lijnen indien meerdere posities overeen komen) wordt door de kernel 'getMinValueIndex' terug meegegeven naar de CPU.

In onderstaande afbeelding kunnen we de output van de console zien. We printen eerst de volledige LUT (hier bevat deze amper 30 elementen, dit zodat deze overzichtelijk blijft ter demonstratie). Nadien kiezen we dus 1 willekeurige lijn, en gebruiken we de 2 kernels om de best passende positie terug te vinden.

```
Lookuptable:
0. |: 0 44 168
1. |: 3 77 42
2. |: 6 11 105
3. |: 0 44 168
4. |: 3 77 231
5. |: 6 110 105
6. |: 9 44 168
7. |: 3 77 231
8. |: 6 110 84
9. |: 9 33 168
10. |: 2 77 231
11. |: 6 110 84
12. |: 9 33 147
13. |: 2 66 231
14. |: 5 110 84
15. |: 9 33 147
16. |: 2 66 231
17. |: 5 110 84
18. |: 9 33 147
19. |: 2 66 210
20. |: 5 99 84
21. |: 8 33 147
22. |: 2 66 210
23. |: 5 99 63
24. |: 8 22 147
25. |: 1 66 210
26. |: 5 99 63
27. |: 8 22 126
28. |: 1 55 210
29. |: 4 99 63

Random line chosen: 10 = 2, 77, 231

Kernel results:
MinValue op plaats: 10
R (verschil dichtst bijzijnde en huidige positie): 0
Index (dichtst passende positie uit LUT): 10
```

Figure 2: Console output

Conclusie

Het doel van ons project hebben we tot een goed einde gebracht. We kunnen gegevens inlezen via het .CSV-bestand en vervolgens met een willekeurige waarde bekijken welke positie overeenkomt. Een eventuele uitbreiding hiervoor zou kunnen zijn om gebruik te maken van shared memory om te kijken of dit de prestaties nog verder zal verbeteren.